

Assignment 9

Aim - To implement ford - fulkerson's algorithm

Analysis - Ford Fulkerson's algorithm is used to find the maximum flow in a flow network.

Steps

- Initialise ~~as~~ all flows to 0
- Find augmenting path using BFS
- Find the ~~maximum~~ minimum capacity on the path
- Update forward and reverse residual capacity
- Add the path flow to maximum flow
- Repeat until no path exists

Time Complexity = $O(V \cdot E^2)$

Space Complexity = $O(V^2)$

V is the number of ~~spaces~~ nodes/vertices in the graph and E is the number of edges.

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define V 10
```

```
int bfs (int graph[V][V], int s, int t, int parent[V]) {
```

```
    int visited[V] = {0};
```

```
    int queue[V];
```

```
    int front = 0, rear = 0;
```

```

• queue[rear++] = 5;
  visited[5] = 1;
  parent[5] = -1;
  while (front < rear) {
    int u = queue[front++];
    for (int v = 0; v < V; v++) {
      if (!visited[v] && rGraph[u][v] > 0) {
        queue[rear++] = v;
        parent[v] = u;
        visited[v] = 1;
      }
    }
  }
  return visited[V];
}

int fordFulkerson(int graph[V][V], int
  source, int sink) {
  int rGraph[V][V];
  for (int u = 0; u < V; u++) {
    for (int v = 0; v < V; v++) {
      rGraph[u][v] = graph[u][v];
    }
  }
  int parent[V];
  int maxFlow = 0;
  while (bfs(rGraph, source, sink, parent)) {
    int pathFlow = INT_MAX;
    for (int v = sink; v != source; v = parent[v]) {
      int u = parent[v];
      if (rGraph[u][v] < pathFlow)
        pathFlow = rGraph[u][v];
    }
  }
}

```

```

for(int u = sink; u != source; u = parent[u]) {
    int u = parent[u];
    graph[u][v] -= pathFlow;
    graph[v][u] += pathFlow;
}
maxFlow += pathFlow;
}
return maxFlow;
}
void main
void main() {
    int graph[10][10] = {
        {0, 16, 13, 0, 0, 6, 0, 0, 0, 0},
        {0, 0, 10, 12, 0, 0, 0, 0, 0, 0},
        {0, 4, 0, 0, 14, 0, 0, 0, 0, 0},
        {0, 9, 0, 0, 20, 0, 0, 0, 0, 0},
        {0, 0, 0, 7, 0, 0, 4, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 10, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 0, 10, 0},
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 15},
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 10},
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
    };
    int source = 0;
    int sink = 9;
    int maxFlow = fordFulkerson(graph, source,
                                sink);
    printf("Maximum Flow = %d\n", maxFlow);
}

```

```
cd "/home/tarun/Downloads/ADSA-TSEC-SEM3/Assignment 9/" && gcc
tarun@Tarun-PC:~/Downloads/ADSA-TSEC-SEM3$ cd "/home/tarun/Dow
ads/ADSA-TSEC-SEM3/Assignment 9/"A1
Maximum Flow = 14
```